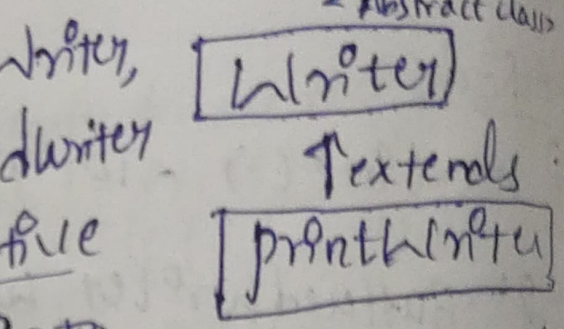
PrintWriter

→ It is class of java.io package.

→ used to write formatted text to an output destination such as a file, console etc.

→ The main advantage of PrintWriter over FileWriter and BufferedWriter is, allows any type of "primitive data" directly to the file, into text format. It then writes that formatted data to writer.



→ Also, the PrintWriter class does not allow to throw any "IO exception". Instead, we need to use the "checkError()" method to find any error in it.

→ Have "Auto-flushing" feature, forces the writer to write all data to the destination if one of the println() or printf() methods is called.

creating printWriter

1. using other writers:

```
FileWriter fileWriter = new FileWriter("output.txt");
PrintWriter pw = new PrintWriter(fileWriter,
                                 autoflush);
```

2. using output streams:

```
FileOutputStream fos = new FileOutputStream("output.txt");
PrintWriter pw = new PrintWriter(fos, autoFlush);
```

3. using filename

```
PrintWriter pw = new PrintWriter(String file, boolean
                                 autoflush);
```

Methods

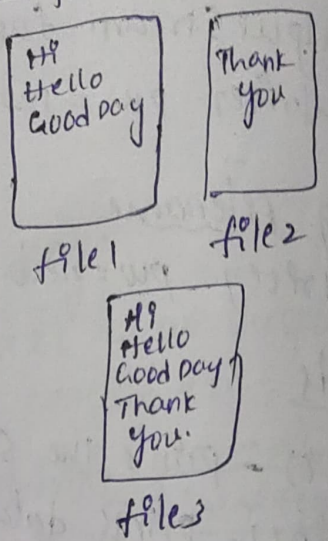
1. print() - prints the specified data to writer.
2. println() - prints data to writer along with newline character at the end.
3. printf() - used to print the formatted string.
4. close() - closes the print writer
5. checkError() - checks if there is an error in the writer and returns a boolean result.
6. append() - appends the specified data to writer.

Example: merging two files of content into 3rd file.

```
import java.io.*;
public class fileMerge {
    psvm(String args[]) {
    PrintWriter pw = new PrintWriter("file3.txt");
    BufferedReader br = new BufferedReader(new
                        FileReader("file1.txt");
    String line = br.readLine();
```

```
while (line != null){
    pw.println(line);
    line = br.readLine();
}
br = new BufferedReader(new FileReader("file2.txt"));
line = br.readLine();
while (line != null){
    pw.println(line);
    line = br.readLine();
}
pw.flush();
br.close();
pw.close();
}
}
```

// output



String formatter

* formatter class provides a more comprehensive way to format strings, numbers and date. It is used internally by String.format() method.

String.format()

→ It is a static method in String class that allows you to format strings using a format string and arguments. The format string contains placeholders for the arguments, which are replaced with the actual values.

Syntax

String formattedString = String.format(format, arguments);

- format : A string with format specifiers (%d, %s, etc)
- arguments : values to insert into the placeholders.

Example:

```
public class StringFormatterDemo{
    psvm(String args[]){
     String name = "John";
     double marks = 95.6765;
      int age = 22;
     String result = String.format("Name : %s, Age : %d,
                     marks : %.2f", name, age, marks);
      sop(result);
      sop(String.format("ID: %05d", 42));
    }
}
```

output:

```
Name : John , Age : 22, marks : 95.67
  ID: 00042  // padding numbers
```

Use cases

→ console or file output formatting
→ Logging readable values
→ Aligning text for tables
→ Creating strings dynamically.

Common format specifiers

Specifier	Description	Example output
1) %d	a decimal Integer	42
2) %f	floating-point (default 6 decimals)	3.141593
3) %.2f	floating-point (2 decimal)	3.14
4) %s	string	Hello
5) %c	character	A
6) %n	newline	line break
7) %05d	Integer with leading zeroes, length 5	00042
8) %-10s	left-align string in 10 spaces	Hello____
9) %o	an octal integer	
10) %x, %X	a hexadecimal integer	
11) %b, %B	Boolean	

flags : used to modify the format of output.

- - left-justify the output
- + always include a sign in output
- 0 pad the output with zeroes
- , includes a comma as a thousand seperator.

→ We can also specify the precision and width of the output using numbers in format specifier.

eg: %.2f

Object serialization and Deserialization

→ java.io package

Serialization

process of converting a java obj into byte stream. The by byte stream can be stored in a file, transferred over a network, or saved to memory.

Use cases

- storing object data to disk
- sending objects over network sockets
- persistent objects in a session or cache..

→ To make a class serializable, it must implement the "Serializable" interface (marker interface - no methods).

Deserialization

→ Reverse process of serialization
→ It involves converting the byte stream back into a copy of original java object.

Example : student.java

```
import java.io.Serializable;
public class student implements Serializable {
    private static final long serialVersionUID = 1L;
    private int id;
    private string name;
    public student (int id, string name) {
        this.id = id;
        this.name = name;
    }
```

```
public String toString() {
    return "ID: " + id + " Name: " + name;
}
}
```

SerializeDemo.java

```
import java.io.*;
public class SerializeDemo {
    public static void main(String args[]) {
        Student s1 = new Student(101, "John");
        try {
            FileOutputStream fos = new FileOutputStream
                                   ("student.ser");
            ObjectOutputStream oos = new ObjectOutputStream(fos);
            oos.writeObject(s1);
            oos.close();
            fos.close();
            SOP("object has been serialized");
        }
        catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

O/P: Object has been serialized.

DeserializeDemo.java

```
import java.io.*;
public class DeserializeDemo {
    p s v m (String args[]) {
      try {
        FileInputStream fis = new FileInputStream("student.ser");
        ObjectInputStream ois = new ObjectInputStream (fis);
        Student S2 = (Student) ois.readObject();
        ois.close();
        fis.close();
        SOP ("Deserialized object: " + S2);
      }
      catch (IOException e) {
        e.printStackTrace();
      }
    }
```

Op: Deserialized object: ID: 101 Name: John.

Serial VersionUID

→ unique Identifier (long type).
→ used during the serialization and deserialization process to verify that the sender and receiver of the serialized object have loaded classes for that object that are compatible wrt serialization.

```
private static final long serialVersionUID = 1L;
```

If that UID not defined,

→ Java will auto-generate it based on the class structure. Even a small change in the class (like adding a file) will cause a different serialVersionUID, leading to: java.io.InvalidClassException.